From Bubble to Next.js in 4 months: the Playgram case study (nano)

The Playgram case study in five minutes — 158 days, 1,029 merged pull requests, and twenty-plus agents in the cloud.

29 August 2026 · 3 min read · Part I of II

Also available: the full version and a mini version. Part I of II. Disclosures approved by Playgram management.

See video at vovazakharov.com/case-studies/assets/playgram-demo.mp4

The job. Playgram is a live AI chat product — many models, team chats, file libraries, memory, voice — built entirely in Bubble, a no-code tool. They wanted it as real code in two months, mainly so they could use AI coding agents on it. I took it, and missed the deadline: the first production build was day 77, all workspaces were over by day 128.

The result. 158 days. 1,395 units of work on `main`, 1,029 merged PRs, 250,000 lines of production TypeScript. Cold loads from multi-second to sub-second. 48 versioned releases and 18 hotfixes — a production deploy every 2.4 days — three workspace cutovers, no rollbacks.

Span	6 March – 10 August 2026 · 158 days
Started at	an 11.6 MB minified JSON — the Bubble app export
Shipped	1,029 merged PRs · 250,000 lines of production TypeScript
Released	48 versioned releases plus 18 hotfixes — a deploy every 2.4 days
Team	four people, and ten to twenty-five Claude Code agents at a time

How. I wrote almost none of it. At the peak, twenty-plus Claude Code agents ran in parallel in the cloud while I reviewed pull requests. Three things made that possible:

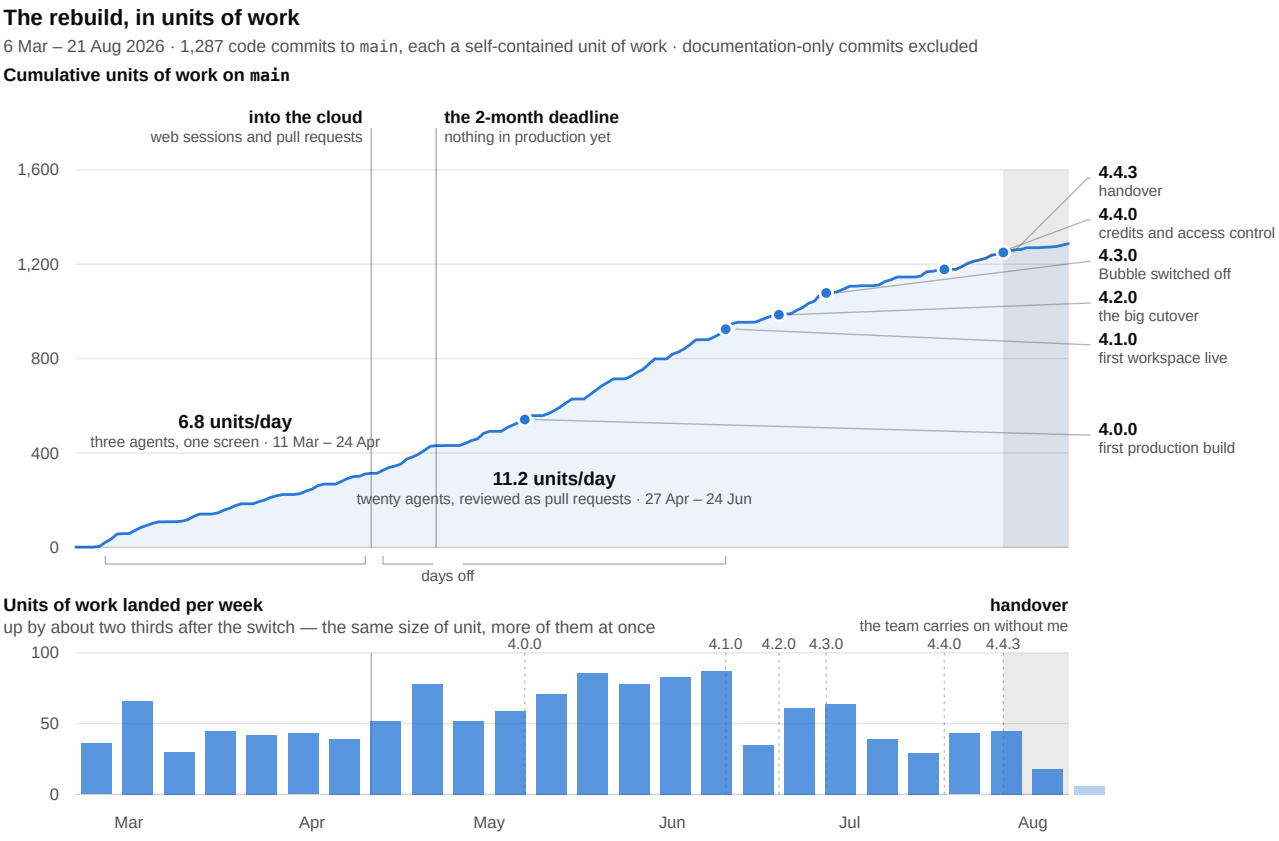
1. **A splitter for the 11.6 MB Bubble export.** Reconstructed by shape rather than chunked by size, into 3,487 files that read almost like source, with names recovered from the export's own fields — and, crucially, stable enough that re-exporting the live app weekly produced a legible diff instead of noise.

2. **Guardrails an agent cannot argue with.** Feature-Sliced Design with zero upward or sideways imports across 8,123 of them, enforced by tooling rather than by good intentions. 362 lint rules, 28 hand-written, every one an error because warnings are rules nobody enforces. Agents will happily ignore a convention written in prose and will never once ship a lint error.

3. **One session, one thread.** Bloated context is the biggest killer of agent productivity and of your budget. Every unit of work gets a plan file in the repo, and a plan good enough to implement is a plan good enough to implement in a *fresh* session — which is the test of whether it was any good.

What that bought. Billing went from a number attached to a price, never enforced, to a metered balance — every reply priced from the real provider cost, decremented live, enforced at send time — in fifteen days. Per-group and per-member model access control took about two. And a per-query CO_2e estimate, asked for by a university customer, went from request to production in eleven days, built by one of the Bubble developers after I'd stopped writing code.

The number I'd put on a slide. Across the switch to parallel cloud agents — one day, 25 April — the median unit of work stayed the same size, about 380 changed lines either side, while units per day went from 6.8 to 11.2, a week off in May aside. Same-sized pieces, two thirds again as many of them at once. That's parallelism, measured.



The part I'd actually point at. I stopped writing code on 10 August. In the seventeen days since, 49 of the 52 pull requests merged were the rest of the team's — and the rest of the team is the three Bubble

developers who built the app in the first place, two of whom opened their GitHub accounts during this project. Every one of those PRs went through the same plan-implement-review pipeline. The scaffolding was as much the deliverable as the app.

What I'd do differently. Less upfront deliberation (four models researching every decision was partly just spreading the blame), and a migration plan with the big picture only — the exhaustively detailed one drifted into a liability within weeks.

Part II: CI/CD, data migration, the things that sound simple and aren't, and why it all seemed ALMOST ready at two months and stayed ALMOST ready for two more.